

Foundations of Data Science for Biologists

Indexing, filtering, and grouping

BIO 724D

2026-09-15

Instructors: Greg Wray, Paul Magwene

Revisiting indexing

Revisiting indexing

Indexing = mechanism to refer to a specific subset of a data object

Square bracket `df[x]` or `df["x"]`

Integer or string argument; returns an *object* (list if indexing a data frame)

Double square bracket `df[[x]]` or `df[["x"]]`

Integer or string argument; returns *values* (atomic vector if indexing a data frame)

Dollar sign `df$x`

String argument; returns *values* (atomic vector if indexing a data frame)

Tidyverse `verb(df, x)`

String argument; typically returns an *object* (list if indexing a data frame)

Take-away lessons about indexing

All four methods of indexing are useful and an essential part of R proficiency

Make your code robust:

- Refer to columns by name, not position

- Refer to rows by condition (or use unique IDs); avoid numerical indexing

- Avoid using ranges

Remember to input column names in different ways for different methods:

- Use quotation marks for `obj["x"]` and `obj[["x"]]`

- Do not use quotation marks for `obj$` and `verb[obj, x]`

Remember that different methods return data in different forms:

- `obj["x"]` and `verb[obj, x]` return lists

- `obj[["x"]]` and `obj$x` return atomic vectors

Revisiting pipes

Passing information in pipes

A function will usually work in a pipe if the first argument refers to the data

Nearly all Tidyverse functions: `select()`, `filter()`, `ggplot()`, etc.

Many base R functions: `length()`, `mean()`, `min()`, etc.

However, not true for some common base R functions: `lm()`

Using functions in pipes:

Using a function by itself: `funcA(df, x)`

Using a function in a pipe: `df |> funcA(x) |> funcB(y)`

Working with pipes

To avoid frustration, keep the following differences in mind when using pipes:

Base R functions:

- Usually expect a **vector** as input

- Usually **do not** accept **NA** values

TidyVerse functions:

- Almost always expect a **data frame** as input (occasionally a **column** or **list**)

- Almost always **do** accept **NA** values

Filtering

Filtering

Filtering subsets rows according to a condition

Anything that evaluates to **TRUE** or **FALSE** is a valid condition

R provides a lot of flexibility in constructing conditions:

- Evaluate data of any kind: numeric, logical, string, set-based, etc.

- Many different operators: **==**, **!=**, **>**, **<=**, **%in%**, etc.

- Can be simple or compound: **&** , **|** logic operators, brackets indicate order

- Can apply to data in one column or multiple columns

- Can include embedded functions

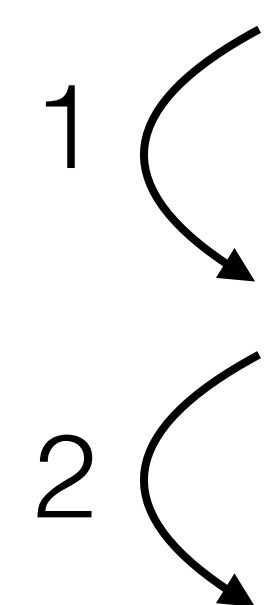
Revisiting Boolean indexing

Step 1: create an index column based on a data column and a condition

```
my_df <- mutate(my_df, my_b_index = my_numbers > 3.0)
```

Step 2: filter based on TRUE/FALSE values

```
my_df |> select(my_numbers) |> filter(my_b_index)
```



1	my_numbers	1.4	3.1	2.7	5.6	4.9	2.2	0.9	3.4
	my_b_index	FALSE	TRUE	FALSE	TRUE	TRUE	FALSE	FALSE	TRUE
2	Returns:	3.1, 5.6, 4.9, 3.4							

Advantages: simpler code, faster execution

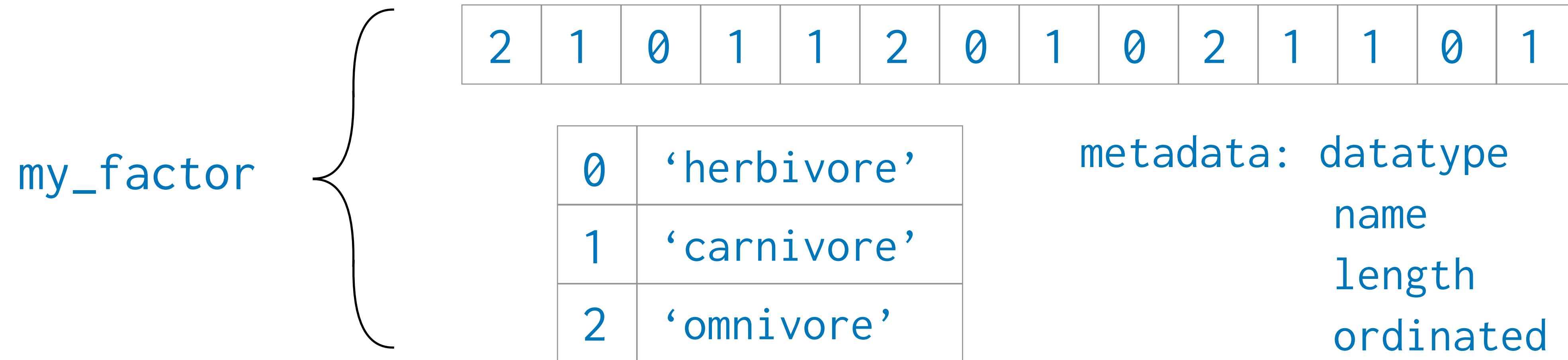
Factors

How do factors work?

Factors are structures with two data components, plus metadata

Integer vector that stores the data

Look-up table that maps integers to strings (optionally ordinated)



Factors provide the best of both worlds

For humans: displayed as strings, thus simple to understand

For computer: stored as integers, thus simple to manipulate

Working with factors

Factors are categorical data vectors

Useful when there is a clearly defined set of allowable values

Contributes to data integrity: will throw an error if a value is not explicitly allowed

In R, allowable values are called **levels**

By default, levels are not ordered (e.g., herbivore, carnivore, omnivore)

Optionally, levels can be ordered (e.g., egg, larva, pupa, adult)

Columns can be coerced between string and factor:

```
as.character(df$fac_column)
```

```
as.factor(df$str_column)
```

Why use factors?

Memory-efficient: occupies much less space than a character vector

Computationally efficient: filter, sort, group_by, etc. are *much* faster

Minimizes data entry errors: reports any unexpected values

Required by some functions and 3rd-party packages

